

Fiendish Photos – Stealing Crypto Seed Phrases with OCR

Author

josh

Date

2025-04-13

Overview

In a previous article, we discussed the powerful ways in which malware authors attack crypto users, from address-swapping attacks, seed-stealing, cryptojacking, and ransomware. This article will focus on a particularly interesting variation of seed-stealing attacks – one that scans *photos* for juicy seed-phrase rewards. In particular, we'll discuss a real-world example of this malware called SparkCat, along with a simple educational demo to break down this attack written in Python.

OCR & Regex – Powerful Pattern Matching

What is OCR and Regex?

For starters, let's define Optical Character Recognition (OCR). This tool allows computer software to scan images such as photographs or document scans for alphanumeric text, and transcribe that text into a format more readily consumed for other purposes – think document editing, textual searches, or transcription. OCR is a fantastic tool for searching photo libraries for text strings, transcribing photocopies, or even allowing screen readers to aid people with accessibility needs.

Regular Expressions (often abbreviated as regex), are another software tool for text scanning and manipulation. Regex is a sort of special-purpose pattern recognition programming language. While much more limited than a full general-purpose programming language, the regex toolkit excels at one task – matching patterns in text strings. For example, it's quite trivial for a programmer to write a regular expression to match the pattern of an email address. A regex can easily determine that someone@theirserver.net is a valid email address, while “somepersonsmail@foo” is not. Regexes are commonly used for matching patterns such as phone numbers, email addresses, and credit card numbers. If you've ever entered a value into a web form field and received a message like “not a valid email address”, that code was likely using regular expressions for that check.

A Demo of Seed-Scanning Malware

Considering how much we use images and text in modern computing, OCR and regex make navigating transcription and pattern matching more accessible for developers and users alike. However, they can also be used for malicious purposes. Seed-scanning crypto malware can use regular expressions to scan various text-based formats for the pattern of seed phrases. Text files, Word Documents, and Open Document Format files are all vulnerable to this sort of attack. This

article won't dive too deep into the development side, but this snippet of Python shows how the BIP39 wordlist can be used to construct a regex for this attack:

```
with open(self.WORDLIST_FILENAME) as f:
    words = [word.strip() for word in f]

combined_words = "|".join(words)
regex_str = r"(" + combined_words + r")\s*">{12,24}"
regex = re.compile(regex_str, flags=re.IGNORECASE|re.MULTILINE)

return regex
```

This snippet uses a file with all the BIP39 words to construct a rather large but effective regex that will match a 12-24 word seed phrase. This snippet, copied from [SeedScanner](#), belongs to my open-source demo of OCR seed-stealing code called [OCRSeedSwipe](#). Note that this regex does not validate if this string is actually a valid BIP39 phrase rather than just 12-24 words from the dictionary. But for an attacker, this is a great starting point. Regex allows for fast pattern matching, and the code could later be adapted to validate if the matched words actually form a crypto seed.

Now that we understand the underlying code used to scan text for seed phrases, we can integrate OCR into the attack. In Python, we can use the popular PyTesseract module. Tesseract is a storied OCR toolkit that has gone through several iterations since the 1980's. Several versions of Tesseract use rule-based pattern matching algorithms to detect text within images. Some modern versions use neural networks, a form of artificial intelligence, to enhance detection capabilities. In our demo, it's a straightforward two-line snippet of code that will pull potential seed words from an image:

```
# Set the Tesseract executable path if necessary, on Windows
pytesseract.pytesseract.tesseract_cmd = self.TESSERACT_PATH

# Scan the image for the words
words = pytesseract.image_to_string(image_filename)
```

A simple unit test that exercises this functionality shows how our library can efficiently extract a seed from a test image. Given that AI is often used for OCR enhancement, I tied some more AI-based learning into this exercise by writing this test with Copilot's assistance.

```
# Copilot Prompt: Write a test that scans "sample_seed.jpg" and
asserts the seed is equal to SEED1
# This test scans the sample seed image and checks it against the
defined test seed
def test_scan_sample_seed(self):

    # Initialize the scanner
    scanner = ocrseedswipe.OCRSeedSwipe()

    # Scan the sample seed image
    target = os.path.join(self.DIR, "sample_seed.jpg")
```

```
seed = scanner.scan(target)

# Assert the seed matches the known seed
self.assertEqual(seed, self.SEED1)
```

With these two tools, we have an interesting and engaging demo of how OCR and regex can be used to scan images and extract seed phrases. While this isn't intended to be a deep development dive, these code snippets demonstrate how efficiently modern coding tools can be used to implement such functionality. A real attacker might employ a more sophisticated approach – precisely what we'll discuss next.

SparkCat Malware

This technique for seed theft is quite fascinating, even if it were purely theoretical. However, this demo is based on techniques employed by real-world malware authors. A recently-discovered attack, dubbed *SparkCat* by security researchers, has targeted users of cryptocurrencies. A detailed [technical report](#) by Kaspersky researchers breaks down the mechanics of this attack.

Both Android and iOS apps were targeted with this malicious code, snuck into otherwise innocuous applications for food delivery and chats. SparkCat makes use of the Google ML Kit library for OCR on both platforms. This functionality is used to scan images for text. The malware's command-and-control server indicated keywords of interest to the attackers, presumably those matching seed words in the BIP39 dictionary. Any images matching these criteria were encrypted and exfiltrated to the attacker's server for further examination. And by further examination, I mean finding seed phrases and using them to steal all of the victim's cryptocurrency.

SparkCat Fire Prevention

This dive into seed-stealing malware with OCR reminds users of a key factor in wallet security – never store a seed phrase in any plain-text form on any device. In this context, plain-text also includes images! In cryptographic terms, plaintext refers to unencrypted data – which is precisely what we want to avoid when storing key material such as a seed phrase. It doesn't matter if your seed is stored in a literal plain text file (.txt), a Word Document, or a screenshot. If it's not encrypted, malware can find it. There's a wide variety of programming tools that make bulk text and image scanning all too easy for an attacker.

Instead, choose to store your seed phrases in a secure fashion. For hot wallets, it's acceptable to store a copy of your seed phrase in an encrypted form, provided that the passphrase used for encryption is long and strong. A hot wallet stores your encrypted keys protected by a passphrase or PIN in the first place. Note that this does add another point of failure, but the overall level of security is roughly the same. A better method for most users is to store the seed written on paper or etched into metal. This is a requirement for hardware wallets, which generate and store the keys offline. You must ensure that a hardware wallet seed is kept offline, away from networked devices such as phones or PCs.

Crypto key security requires a thoughtful approach to potential avenues of compromise. Attacks against crypto users grow in sophistication all the time, and so users must stay up-to-date on security best practices. Keep those seeds safe, and protect your stack from malware threats.